Made with Goodnotes

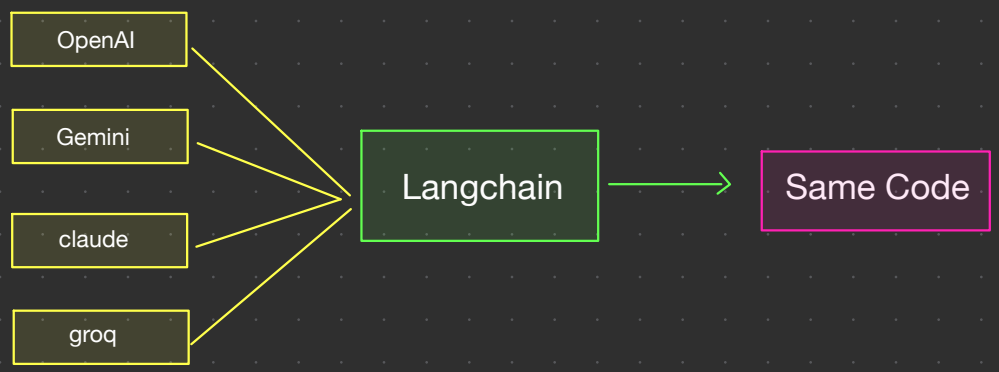
Runnables

So imagine you want to build a chatbot. For that, you obviously need an LLM.
Now earlier, there was a big problem. There were many different LLM providers like OpenAI, Anthropic, Google, and others. The issue was that each of them had their own SDK, their own syntax, and their own way of calling the model.

Because of this, if you built your project using one provider, like OpenAI, and later decided to switch to something like Gemini or Claude, you basically had to rewrite a large part of your code. That was frustrating and time-consuming.

This is where LangChain comes in and solves the problem beautifully.

LangChain provides a unified interface. That means no matter which LLM you use, the way you write your code remains almost the same. You just change the model, and everything else stays as it is.
This flexibility is one of the biggest reasons why developers love LangChain. It gives you the freedom to experiment and switch between models without rebuilding your entire project from scratch.



So once developers started using LLMs, things got interesting very quickly.

At first, it was simple. You had a model, you gave it some input, and it gave you an output. That was it.
But as people started building real applications, they realized something important a single LLM call is not enough.
Let's say you're building something like a chatbot, or a document Q&A system, or even a summarization tool. Now your application is not just one step anymore. You might need to first format the input properly, then send it to the model, and then clean or structure the output before showing it to the user.
So now instead of one step, you suddenly have multiple steps.
You have:

- • some kind of input processing
- • then a prompt template
- • then the LLM call
- • and then an output parser

At this point, you can clearly see that your application is no longer just a single model call.
Now you have multiple components involved things like LLMs, prompt templates, output parsers, and even custom logic in between.
And all of these components need to work together in a proper sequence to give you the final result.

Made with Goodnotes

Some of the most common ones are:

- • **LLM / Chat Model** → the core model that generates responses
- • **Prompt Template** → formats the input before sending it to the model
- • **Output Parser** → cleans or structures the model's response
- • **Retriever** → fetches relevant data (used in RAG systems)
- • **Document Loader** → loads data from PDFs, files, etc.
- • **Text Splitter** → breaks large text into smaller chunks
- • **Embeddings Model** → converts text into vectors
- • **Vector Store** → stores and searches embeddings

```python
from dotenv import load_dotenv
load_dotenv()

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Model
model = ChatOpenAI(model="gpt-4o-mini")

# 2. Prompt Template
prompt_template = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words"
)

# 3. Output Parser
parser = StrOutputParser()

# Step-by-step manual flow (NO chains)

# Format the prompt
formatted_prompt = prompt_template.format_messages(topic="Machine Learning")

# Call the model (old style)
response = model.predict_messages(formatted_prompt)

# Parse the output
final_output = parser.parse(response.content)

print(final_output)
```

The base code
We and everyone
else used
to crete a basic
chatbot

So even though we now have better components like prompt templates and output parsers…
we are still manually connecting everything step by step.

So now if you look at this, even though we are using better components like prompt templates, models, and output parsers… we are still doing everything manually.
We have to:

- • format the prompt
- • call the model
- • parse the output

Now imagine you are building a large application, something like a RAG system.
In that case, you're not just dealing with one or two components anymore. You have a retriever, a prompt template, an LLM, an output parser, maybe even embeddings and a vector database.
And the problem is… you have to call each of these components manually.
First you retrieve the data, then you format the prompt, then you call the model, then you parse the output… and you have to manage the flow between all of them yourself.
As your application grows, this quickly becomes messy and difficult to maintain.
Handling so many moving parts manually is not just repetitive, it's also error-prone.

Made with Goodnotes

```python
from dotenv import load_dotenv
load_dotenv()

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Model
model = ChatOpenAI(model="gpt-4o-mini")

# 2. Prompt Template
prompt_template = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words"
)

# 3. Output Parser
parser = StrOutputParser()

# 🔥 Chain (everything connected automatically)
chain = prompt_template | model | parser

# Run the chain
result = chain.invoke({"topic": "Machine Learning"})

print(result)
```

So chains were definitely a big improvement.
They made our code cleaner, reduced repetition, and allowed us to connect multiple components into a single pipeline.
But as developers started building more complex applications, a new problem started to appear.
There wasn't just one type of chain.
LangChain had introduced many different chains for different use cases.
For example:

- • **LLMChain** → for simple prompt + model tasks
- • **SequentialChain** → for multi-step workflows
- • **RouterChain** → for dynamic decision making
- • **RetrievalQA Chain** → for RAG-based applications

Now on paper, this sounds useful.
But in practice, it became confuing.
As a developer, you always had to think:
"Which chain should I use for this problem?"
And sometimes, your use case didn't fit perfectly into any one chain.
So you either had to:

- • force your problem into an existing chain
- • or combine multiple chains in a complicated way

This made things harder instead of easier.
On top of that, chains were not very flexible.
If you wanted:

- • custom logic in between steps
- • parallel execution
- • or more control over how data flows

Chains started to feel restrictive.
So instead of simplifying things, the growing number of chains actually created confusion and friction for developers.
And because of all these limitations, LangChain made an important decision.
They moved away from the concept of predefined chains and introduced a much more flexible system… Runnables

Made with Goodnotes

Now this is where things become really interesting.
Instead of having different types of chains for different use cases, LangChain introduced a much simpler and more powerful idea.
They said what if every component we are using is treated the same way?
Think about it.
We already have multiple components in our application:
we have prompt templates, we have models, we have output parsers, retrievers, and even custom functions.

Now instead of treating all of these differently, LangChain introduced a unified concept…
Everything is a Runnable.
That means:
your prompt template is a runnable,
your model is a runnable
your output parser is a runnable,
even your own Python function can be a runnable.
So instead of thinking in terms of different types of chains, now you just think in terms of **runnables that can be connected together**.

Now imagine all these components as **Lego blocks**.
Each block has its own purpose. One block might be a prompt template, another might be a model, another could be an output parser, and so on.
Individually, they all do different things.
But here's the interesting part…
No matter what the block does, the way you connect one block to another is always the same.
Just like Lego.
You don't need a different way to connect different pieces. Every block follows the same connection structure.
And that's exactly what LangChain did with runnables.

Now there's one really interesting thing about runnables.
All these different components, whether it's a prompt template, a model, an output parser, or even your own function, they all expose the same kind of methods.
For example, you've already seen one of them:
👉 **invoke()**
This is used to run the component on a single input and get the output.
But that's not the only one.
You also have:
batch() when you want to process multiple inputs at once
stream() when you want to get output token by token like real-time responses

Now think about this…
Completely different components, doing completely different tasks, but all of them can be used in the exact same way.
And that is the real power here.
You don't need to learn different ways to use different components. Once you understand how one runnable works, you can use any other runnable the same way.

Made with Goodnotes

Actual Runnables

These are your components/runnables:

- • LLM
- • Prompt Template
- • Output Parser
- • Retriever
- • Tools

Basically: *they do the actual job*

Runnable Sequence

Now that we understand what runnables are, let's see how we actually use them in real applications.
We're going to learn this in three simple steps.
First, we'll start with the most basic and most important pattern a simple sequence.
In this, we just connect components like a pipeline. The output of one component becomes the input of the next. So your flow becomes something like prompt to model to parser. This is the foundation, and honestly, most applications start like this.

Parallel Runnable

Now till this point, we have only seen how to build a single pipeline where the input flows step by step and produces one output. But in real-world applications, sometimes we don't want just one result. We might want multiple outputs at the same time, like a short explanation and a detailed explanation, or maybe the same answer in different formats. Instead of running the pipeline again and again for each case, we can use a Parallel Runnable.

In this approach, we define multiple pipelines inside a dictionary, and all of them run simultaneously on the same input. Each pipeline produces its own output, and in the end, we get all the results together in a structured form. So instead of one input giving one output, now one input can give multiple outputs in a single execution, which makes our applications more powerful and efficient.

Runnable Lambda

Now sometimes in our pipeline, we don't just want to pass data forward, we might want to slightly modify it or pick a specific part of it before sending it to the next step. This is where RunnableLambda comes in. It allows us to write a simple Python function and insert it inside our pipeline as a runnable. For example, if our input is a dictionary with multiple keys, and we only want to send one specific part of it to a particular pipeline, we can use RunnableLambda to extract that part. So instead of the entire input going everywhere, we can control exactly what each component receives. In simple terms, RunnableLambda lets us add custom logic inside our runnable flow, making our pipelines more flexible and powerful.

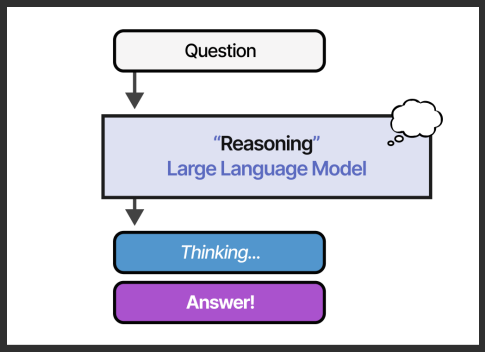
Made with Goodnotes

Runnable Passthrough

Runnable Passthrough is used when we want to keep the original input or some intermediate data while passing it through the pipeline. Normally, in a sequence, each step replaces the previous output, so earlier data gets lost. But in many real-world scenarios, we need to carry multiple pieces of information together. RunnablePassthrough allows us to forward the input as it is, without modifying it, so that it can be used along with other outputs in later steps. In simple terms, it helps us preserve data while still continuing the flow of the pipeline.

Tools

An LLM (Large Language Model) is an AI system trained to understand and generate human language.



Core capabilities an LLM can think / reasoning
And then generate an answer.

But our LLMs are not good with many things like

- No Real-Time Data
- Not Reliable for Calculations
- Cannot Call APIs
- Cannot Access Your Data

Cannot Perform Actions

Examples:

- • Send email ❌
- • Book ticket ❌
- • Run code ❌
- • Update database ❌

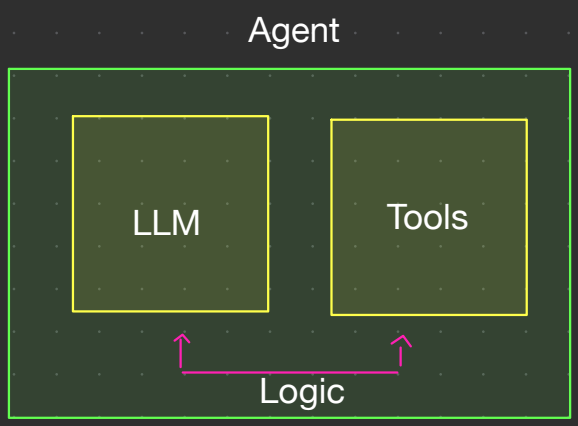
Tools are simply **functions or external services that an LLM can use to do real-world tasks**. On its own, an LLM can only generate text, but with tools it can **fetch live data, perform accurate calculations, access databases, or execute actions**. In short, tools extend an LLM from just *thinking* to actually *doing things*.

Made with Goodnotes

We use an LLM as the brain, tools to perform tasks, and our code to define the logic together, this creates an agent.

LLM decides *what to do*,
tools help *do the task*,
and our code controls *how everything works together*.

Agent = Brain (LLM) + Tools + Control Logic (our code)



Tools category

Build-in Tools

Built-in tools are tools that are already provided by a framework, so you don't have to create them from scratch. They are ready to use and help the LLM perform common tasks like searching the internet, doing calculations, or working with data. You just connect them to your LLM, and it can start using them easily.

Custom tools

Custom tools are tools that **you create yourself** based on your needs. They are usually simple functions written in code that help the LLM perform specific tasks, like accessing your database, processing your data, or calling your own APIs. This allows you to give the LLM **exact abilities that match your project**.

Made with Goodnotes

Tool Binding

Tool binding means connecting our tools to the LLM so it can use them when needed.

We give our tools to the LLM, and now the LLM knows:

- • what tools are available
- • what they do
- • when to use them

Tool calling

In tool binding, we gave tools to the LLM…
in tool calling, the LLM chooses to use them.

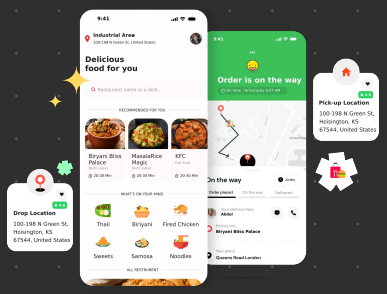
The LLM does NOT execute the tool it only suggests the tool call.

Tool Execution

Tool execution is when we actually run the tool after the LLM decides to use it.
LLM selects the tool → we execute it → get the result → send it back to LLM.

AI agents

So let's say today is Sunday and you want to go on a date with your girl/boy friend now what are you going to do.



You might:

- • Check weather 🌦️
- • Order food 🍔
- • Book a cab 🚗
- • Look for a movie 🎬

You are not doing one task… you are making *decisions* and using different tools.

Made with Goodnotes

Title:

Now Imagine AI Doing This
Content:
User says:
"Plan my evening"
AI should:

- • Understand intent
- • Decide steps
- • Use tools
- • Give final result

💡 This is where **Agents** come in

Agent = Brain + Tools + Decision Making

Lets Create an Agent

We are going to create a city intelligence Agent and we need some tools for that

1. Weather Tool
API: OpenWeatherMap
Purpose:

- • Get **real-time weather data**

Why it's needed:
LLM does NOT know live weather → needs external tool

2. News Tool
API: Tavily
Purpose:

- • Get **latest news about a city**

Why it's needed:
LLM does NOT know current events → needs search

User query
↳ LLM Decides
→ weather? → Call weather tool
→ News? → Call news tool
↓
Tool result
↓
LLM formats the final answer

Made with Goodnotes

In our code, we didn't just call an API or use an LLM separately. We created a system that could understand a user's intention, decide what action is needed, execute that action using external tools, and then use the result of that action to generate a meaningful response. For example, when the user asked about weather, the LLM first analyzed the question and decided that weather information is required. Then it triggered the weather tool, which fetched real-time data from an API. After that, we fed that result back into the LLM, and only then the LLM generated the final answer.
So what we really built was not just code we built a decision-making loop where the LLM acts like a brain and the tools act like hands interacting with the real world.

In your manual code, every single step is visible:

- • when the LLM is called
- • when a tool is triggered
- • what data is passed
- • what result comes back

You can literally trace the entire flow from user input to final output.

But there are some problems with this code I am not saying the code is bad actually its really good but lets see some problems

Too Much Boilerplate Code

In your implementation, you had to write:

- • the loop
- • tool execution logic
- • message handling
- • condition checks

All of this is **repetitive work**.

Manual Loop Management is Error-Prone

You are responsible for:

- • when to stop the loop
- • when to continue
- • handling edge cases

A small mistake like:

- • forgetting `continue`
- • misplacing a condition

👉 can break the entire agent.

Scaling Becomes Difficult

Right now you have:

- • 2 tools (weather + news)

Imagine:

- • 10 tools
- • multiple conditions
- • complex workflows

Your code will quickly become:

- • messy
- • hard to maintain
- • difficult to extend

Harder to Maintain Over Time

As your project grows:

- • more tools
- • more features
- • more logic

The manual loop becomes harder to manage.

Made with Goodnotes

`create_agent` is a high-level abstraction provided by LangChain that automates everything we manually implemented in our agent. Instead of writing the loop, handling tool calls, managing messages, and deciding when to stop, we simply define the model and the tools, and the framework takes care of the entire execution process internally. The core logic remains the same the LLM still decides which tool to use, tools still fetch external data, and results are passed back for final response generation but all of this happens behind the scenes. The main difference is that our manual approach gives full control and visibility over each step, while `create_agent` hides this complexity to make development faster, cleaner, and easier to scale.

Made with Goodnotes